TEXAS TECH UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
CAPSTONE PROJECT LABORATORY

Infrastructure-Free Vehicle-to-Vehicle Tracking: Design and Analysis of a LoRa Position Link Between Two Heltec WiFi LoRa 32 V4 Nodes with Dedicated Dashboard Displays

A Technical Report on the Off-Grid Vehicle Tracker

Group 9
Sponsor: Dr. Richard Gale
Fall 2026

Abstract

This report explains how two vehicles track each other with no cellular network, no internet, and no infrastructure of any kind between them. Each vehicle carries a Heltec WiFi LoRa 32 V4 radio node running the open-source Meshtastic firmware and a Waveshare ESP32-S3 reflective-LCD dashboard display running firmware written by this team. The radios learn their own positions from GNSS satellites and broadcast them to each other over a 915 MHz LoRa link; each display receives the decoded traffic from its local radio over Bluetooth Low Energy and converts it into a live navigation screen showing the partner's direction, distance, and speed. The report develops the complete chain that makes this possible: the chirp spread spectrum physical layer that decodes signals 17.5 dB below the noise floor, the time-on-air arithmetic that prices every packet at roughly 0.56 seconds, the 161 dB link budget that supports a 3 to 5 km design range, the Meshtastic channel contract in which a hashed channel name becomes the operating frequency, and the published Bluetooth Client API that allowed a custom display to attach to unmodified radio firmware. All claims are tied to measurements made on this hardware, and all mathematics is worked with this system's actual parameters.

1. Introduction

Modern vehicle tracking assumes infrastructure. A phone reports its position through a cell tower; a fleet tracker reports through a satellite service; both stop working exactly where tracking matters most, on desert trails, in mountain valleys, and on rural highways beyond coverage. The goal of this project is a system in which two vehicles always know each other's position using nothing but the equipment they carry. There are no accounts, no subscriptions, and no third party between them.

One idea organizes this entire report: in an infrastructure-free radio system, *agreement is the connection*. There is no session, no pairing, and no handshake between the two radios. They exchange data exactly when they independently agree on three things: the frequency they occupy, the modulation they speak, and the encryption key they share. Every design decision in the following sections either establishes one of those agreements, budgets the physical resources that keep the agreement usable, or displays the agreed information honestly to a driver. Section 2 defines the architecture. Sections 3 through 5 build the radio node and its physical layer from the datasheet upward, in the same order the signal itself is built. Section 6 explains the protocol layer that turns raw packets into a private, self-managing channel. Sections 7 and 8 cross into the display node and the navigation mathematics it computes. Section 9 reports the experimental verification, and Section 10 concludes.

2. System Architecture

The system consists of four embedded devices arranged in two identical halves, one per vehicle. Each half contains a radio node and a display node.

Device	Hardware	Firmware	Role
Radio node (2)	Heltec WiFi LoRa 32 V4	Stock Meshtastic 2.7.26	Acquires its own GNSS fix; broadcasts position, telemetry, and text over LoRa; serves decoded traffic to its display over BLE
Display node (2)	Waveshare ESP32-S3-RLCD-4.2	Custom, written by Group 9	Connects to its local radio over BLE; decodes the protocol stream; renders direction, distance, speed, messages, and warnings

Figure 1 shows the complete signal path. Information flows in one direction around a loop. Vehicle A's radio measures its position and broadcasts it over LoRa; vehicle B's radio receives and decrypts the packet and holds it in a node database; vehicle B's display drains that database over Bluetooth Low Energy and renders the result. The same path runs simultaneously in the opposite direction. The deliberate consequence of this architecture is that the two engineering problems are separable: the radio link is a solved, heavily tested open-source system that this team configures and characterizes but does not modify, while the display is entirely this team's code attached at a published interface. Section 7.2 examines that interface, because it is the single decision that made the project feasible in one semester.

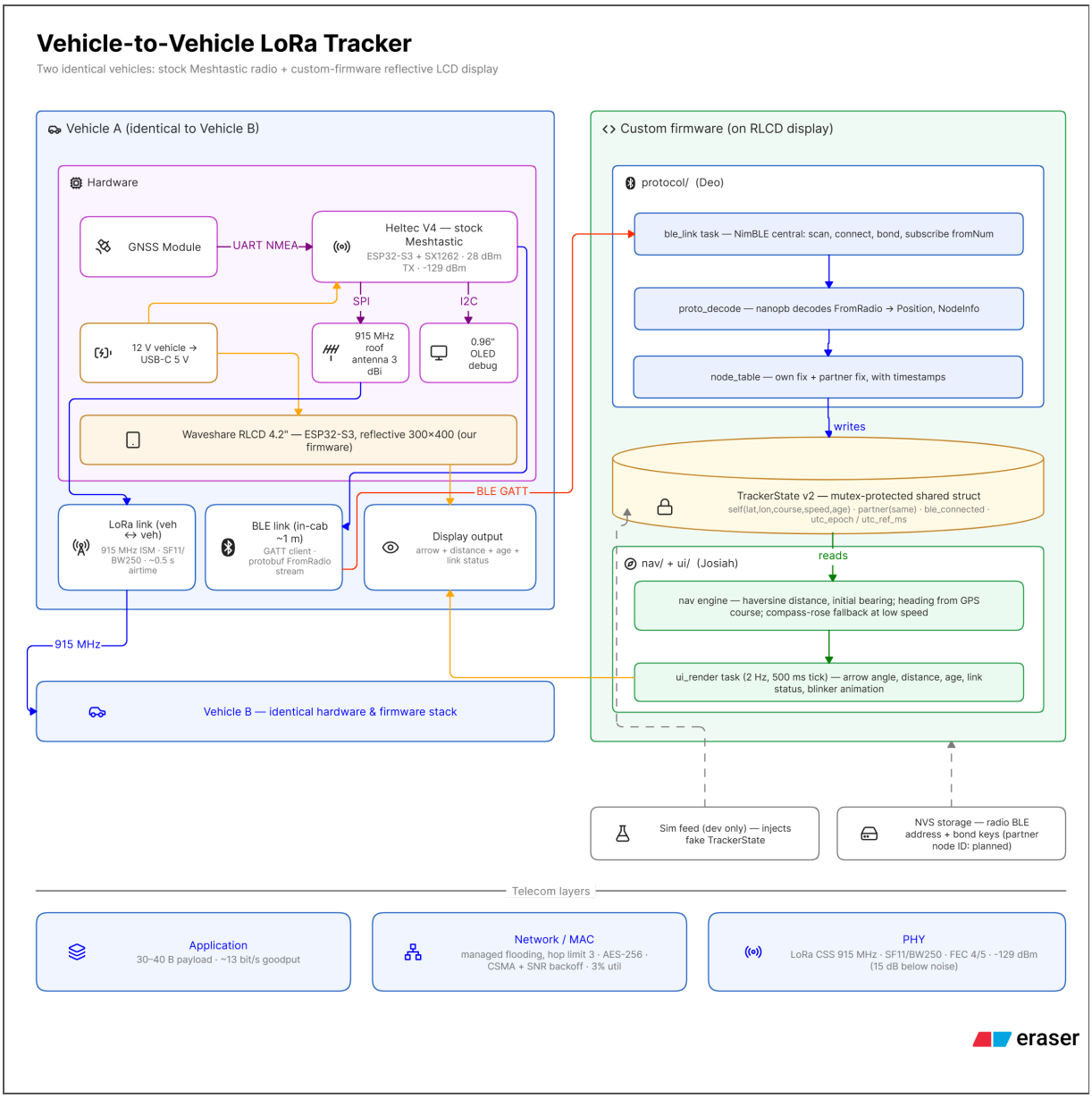


Figure 1. System architecture. Each vehicle pairs a stock Meshtastic radio with a custom-firmware reflective display; the vehicles share only the 915 MHz LoRa channel.

3. The Radio Node: Heltec WiFi LoRa 32 V4

The Heltec V4 is a development board that integrates the three subsystems a tracking node needs. An Espressif ESP32-S3 microcontroller, a dual-core Xtensa LX7 at 240 MHz with 8 MB of flash, runs the Meshtastic firmware and its FreeRTOS task set (Espressif). A Semtech SX1262 transceiver provides the sub-GHz LoRa physical layer; it is a half-duplex radio covering 150 to 960 MHz, programmable in spreading factor, bandwidth, and coding rate, with a receive sensitivity that reaches -131 dBm at the settings used here (Semtech, *SX1261/2*). A u-blox GNSS module supplies position fixes, typically converging to a five-satellite fix within a minute under open sky. The board carries a 0.96-inch OLED status screen, a USB-C port for provisioning, and a lithium cell for operation away from vehicle power.

3.1 Inside the transceiver

Figure 2 reproduces the SX1262's internal block diagram from the datasheet, and it rewards a careful walk from left to right because the whole receive path of Section 4 lives inside it. The antenna enters through a matching network and low-pass filter. On receive, a low-noise amplifier lifts the faint arriving signal, a mixer driven by the phase-locked loop translates it down from 913 MHz, and an analog-to-digital converter hands the result to one of two demodulators, the LoRa modem used here or a conventional FSK modem. The protocol engine assembles demodulated bits into packets in the data buffer, which the ESP32-S3 reads over a four-wire SPI bus. On transmit the path runs in reverse, ending in the power amplifier. The crystal oscillator and PLL that set the carrier frequency are shared by both directions, which matters in Section 6: when Meshtastic computes a frequency from the channel name, this PLL is the hardware that must be programmed to it, and the OLED frequency readout this team treats as ground truth is a report of that programming. The DC-DC converter and low-dropout regulator in the corner of the diagram power the chip, and their efficiency is part of why a node runs for days on a small cell.

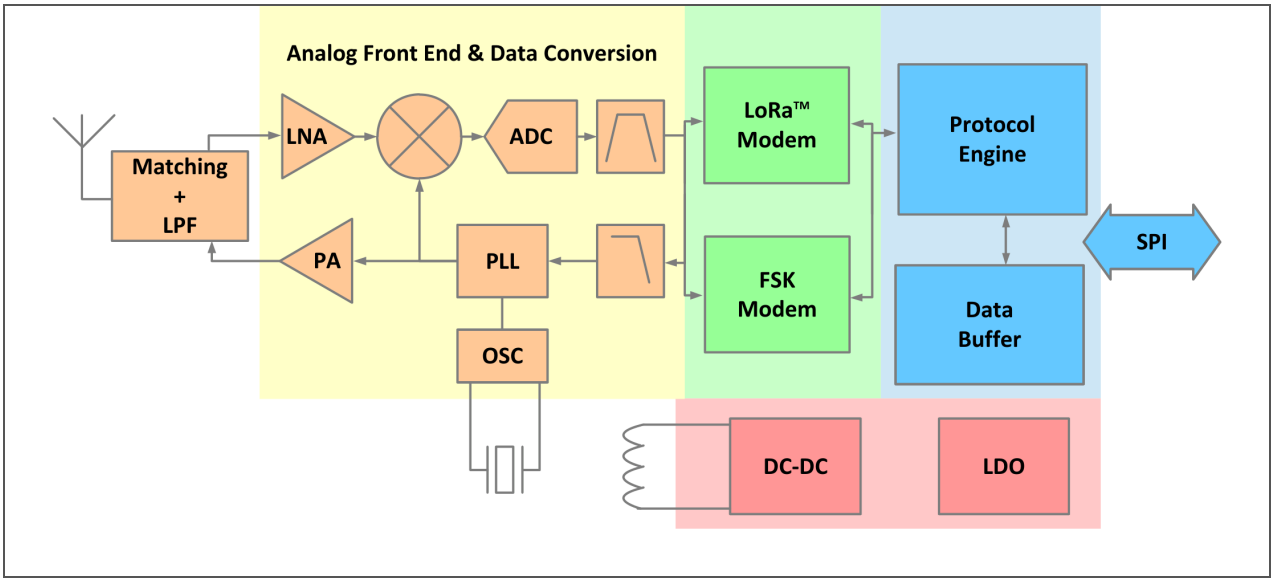


Figure 2. Internal block diagram of the SX1261/2 transceiver, adapted from the datasheet, revision 2.2, figure 4-1 (Semtech, *SX1261/2*).

Three facts from this datasheet shape the whole design. First, sensitivity is not a single number but a function of the modulation parameters; the -131 dBm figure used in Section 5 belongs specifically to the SF11, 250 kHz operating point, so choosing those parameters is choosing that sensitivity. Second, the bare SX1262 tops out at +22 dBm, and the Heltec board adds amplification to reach the +30 dBm the configuration commands, which happens to be the legal ceiling for this band under 47 C.F.R. § 15.247

(United States). Third, the radio is half duplex, and the datasheet's own operating-mode diagram, reproduced in Figure 3, shows why.

3.2 The transceiver state machine

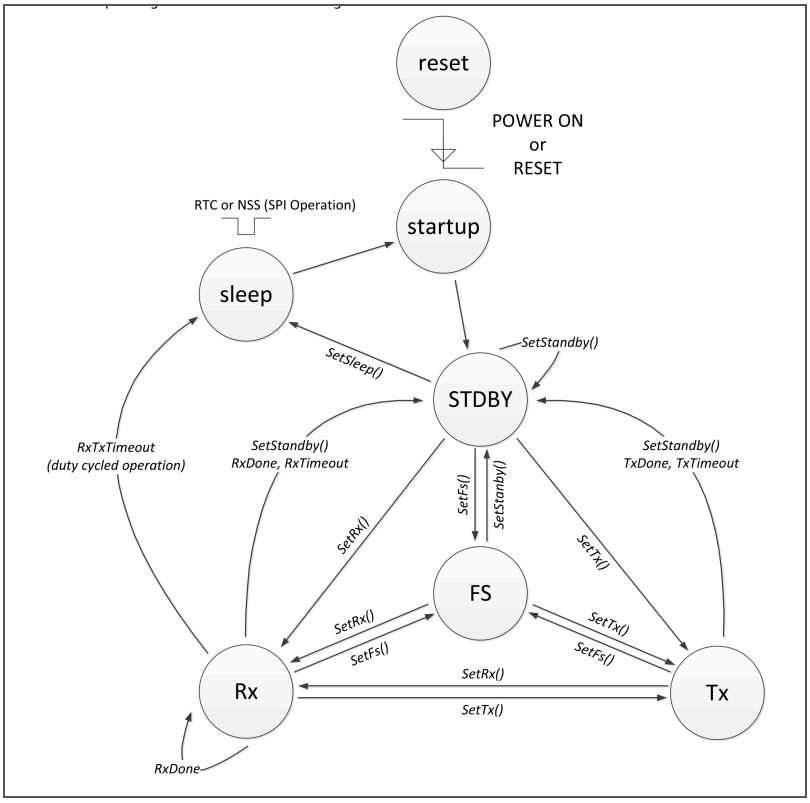


Figure 3. Transceiver circuit modes of the SX1261/2, adapted from the datasheet, revision 2.2, figure 9-1 (Semtech, *SX1261/2*).

The chip is always in exactly one of the circuit modes in Figure 3. From power-on it passes through startup into standby, where the host configures it; the frequency-synthesis mode locks the PLL onto the carrier; and the Rx and Tx states are where the radio actually listens and speaks. The diagram contains no state in which the chip does both, and the Rx and Tx circles are connected only through explicit commands. That single picture is the physical root of a constraint the protocol layer inherits: while a node spends its 0.56 seconds in Tx transmitting a position packet, it is deaf, so two nodes transmitting at once on one channel simply lose each other's packets. Meshtastic manages this with listen-before-talk and randomized backoff, but no protocol can repeal the state machine, which is why Section 6.3 treats airtime as a budget to be rationed rather than a convenience. The sleep state matters too; Meshtastic parks the radio there between duties, which is where the days-long battery figures come from.

4. The Physical Layer: LoRa Chirp Spread Spectrum

4.1 How a chirp carries bits

LoRa (Long Range) is a chirp spread spectrum modulation. Each symbol is a frequency sweep that crosses the entire channel bandwidth, and the information is carried in the sweep's cyclic starting offset. With spreading factor SF, the sweep is divided into 2^{SF} distinguishable offsets, so one symbol carries SF bits. This system uses the Meshtastic LONG_FAST preset: SF = 11, bandwidth BW = 250 kHz, and coding rate 4:5 (Meshtastic Project, "Radio Settings"). A spreading factor of 11 gives $2^{11} = 2048$ offsets and therefore 11 bits per symbol. The duration of one symbol is the chip count divided by the bandwidth:

$$T_{sym} = \frac{2^{SF}}{BW} = \frac{2^{11}}{250{,}000\ \text{Hz}} = 8.19\ \text{ms}$$

The receiver demodulates by multiplying the incoming signal with a locally generated inverse chirp, which collapses each sweep into a single frequency bin recoverable by a Fourier transform. Because this de-chirping integrates all 2048 chips coherently, the processing gain allows the SX1262 to demodulate at a signal-to-noise ratio of -17.5 dB at SF11 (Semtech, *SX1261/2*). The signal is literally quieter than the noise floor, and the receiver still reads it. This is the physical origin of LoRa's range, and its price is time: every symbol lasts 8.19 ms, which makes airtime the scarce resource the rest of the design must budget.

4.2 The cost of one packet

The number of symbols required to carry a payload follows Semtech's time-on-air formula, in which PL is the payload length in bytes, H is zero when an explicit header is present, DE is the low-data-rate optimization flag, CRC indicates the integrity checksum, and CR = 1 encodes the 4:5 coding rate. The numerator counts the bits to move, the denominator counts the bits each symbol carries, and the factor (CR + 4) prices the forward error correction (Semtech, *AN1200.13*):

$$N_{payload} = 8 + \max\left(\left\lceil \frac{8PL - 4SF + 28 + 16\,CRC - 20H}{4(SF - 2DE)} \right\rceil (CR + 4),\ 0\right)$$

Evaluating this for a representative 40-byte Meshtastic position packet at SF = 11, BW = 250 kHz, DE = 0, with the CRC enabled and a 16-symbol preamble, gives 48 payload symbols, and adding the preamble and the 4.25 synchronization symbols yields the total transmission time:

$$N_{payload} = 8 + \left\lceil \frac{8(40) - 44 + 28 + 16}{44} \right\rceil \times 5 = 48, \quad T_{pkt} = (16 + 4.25 + 48)\,T_{sym} = 68.25 \times 8.19\ \text{ms} \approx 0.56\ \text{s}$$

One position update costs roughly 0.56 seconds of air. This single number propagates through the entire system: it sets the fastest responsible reporting cadence (Section 6.3), it explains why the protocol cannot update once per second, and it is the unit in which the channel budget of Section 6.3 is denominated.

5. The Link Budget: From Datasheet Numbers to Street Range

A link budget adds every gain and subtracts every loss between transmitter and receiver, then asks whether what arrives still clears the receiver's sensitivity. The received power is the transmitted power plus both antenna gains minus the free-space path loss, which grows logarithmically with both distance and frequency:

$$P_{rx} = P_{tx} + G_{tx} + G_{rx} - FSPL, \qquad FSPL = 20\log_{10} d + 20\log_{10} f - 147.55 \ \ (d \text{ in m},\ f \text{ in Hz})$$

In this system the transmit power is +30 dBm, the stock quarter-wave antennas contribute approximately 0 dBi each, and the sensitivity at the chosen operating point is -131 dBm, giving a total budget of 161 dB of loss the link can absorb before decoding fails. Evaluating the path loss at the operating frequency of roughly 913 MHz for two reference distances shows how gently it grows:

$$FSPL_{1\,\text{km}} = 60 + 179.2 - 147.55 = 91.7\ \text{dB} \Rightarrow P_{rx} = -61.7\ \text{dBm}; \qquad FSPL_{5\,\text{km}} = 105.7\ \text{dB} \Rightarrow P_{rx} = -75.7\ \text{dBm}$$

Even at 5 km the free-space arrival of -75.7 dBm clears sensitivity by 55 dB. Streets, however, are not free space: terrain, buildings, foliage, and multipath fading consume margin unpredictably, so the design reserves 20 dB for the environment and quotes an honest design range of 3 to 5 km rather than the tens of

kilometers the free-space arithmetic would permit. Margin converts to distance through the doubling rule implied by the 20 log term: every 6 dB of spare margin roughly doubles the achievable range. The measured desk-range baseline of roughly 6 dB signal-to-noise ratio sits 23.5 dB above the -17.5 dB demodulation floor, which is the reserve the system spends as the vehicles separate:

$$\frac{d_2}{d_1} = 10^{\Delta/20} \Rightarrow \Delta = 6\ \text{dB} \approx \times 2\ \text{distance}, \qquad \text{margin} = SNR_{meas} - SNR_{floor} = 6 - (-17.5) = 23.5\ \text{dB}$$

6. The Protocol Layer: Meshtastic and the Channel Contract

6.1 The name is the frequency

The radios run Meshtastic, an open-source LoRa mesh networking firmware (Meshtastic Project, *Firmware*). Its most consequential design choice for this project is that a node stores no frequency at all. The United States band plan spans 902 to 928 MHz, which at 250 kHz per channel divides into 104 slots, and the firmware derives its slot by hashing the primary channel's name (Meshtastic Project, "Radio Settings"):

$$f_{slot} = f_{start} + \frac{BW}{2} + [\text{djb2}(name) \bmod N_{slots}] \cdot BW, \qquad N_{slots} = \frac{f_{stop} - f_{start}}{BW} = \frac{26\ \text{MHz}}{250\ \text{kHz}} = 104$$

The hash is djb2, which starts from the constant 5381 and, for each character, multiplies the running value by 33 and adds the character code. The formula can be checked against a public constant: djb2 of "LongFast," the default channel name, lands on slot 19, which is 906.875 MHz, the documented United States default. Our private channel name "Group9" hashes to slot 43, and substituting into the slot formula gives the frequency both radios must, and verifiably do, display:

$$f_{43} = 902.000 + 0.125 + 43 \times 0.250 = 912.875\ \text{MHz}$$

Two radios holding identical encryption keys but different channel names therefore sit megahertz apart and never exchange a bit. This team learned that lesson experimentally: the single longest outage of the semester was traced to a channel write that silently reverted on one radio, leaving both nodes healthy, both configurations apparently identical, and the pair mutually deaf on different slots. The durable diagnostic that emerged is that the frequency printed on each radio's OLED screen is the ground truth of the agreement, checked before every test.

6.2 Privacy on a public frequency

Slot 43 is public airspace; anyone may transmit or listen there. Privacy comes from the second term of the contract. All channel traffic is encrypted with AES-256 under a pre-shared key generated at channel creation and distributed between the radios only over USB, inside the provisioning URL; the key never crosses the air (Meshtastic Project, "Encryption"). Directed traffic such as on-demand position requests is additionally encrypted per node with Curve25519 public keys exchanged automatically in node-information broadcasts. An eavesdropper observes that packets exist, their timing, and their signal strength, because headers are cleartext, but recovers neither content nor location. The system's privacy is therefore layered exactly like its connectivity: the obscure frequency provides separation from the public mesh, and the key provides the actual secrecy.

6.3 Airtime as a shared budget

Because every packet costs 0.56 seconds of a channel only one node can occupy at a time, Meshtastic treats channel utilization as a managed resource and suppresses position broadcasts when utilization exceeds roughly 25 percent (Meshtastic Project, "Mesh Broadcast Algorithm"). The configured cadence must therefore be shown to fit. With both vehicles reporting every 10 seconds while moving, utilization is the node count times the packet time divided by the cadence:

$$U = \frac{N_{nodes} \cdot T_{pkt}}{T_{cadence}} = \frac{2 \times 0.56}{10} \approx 11\% \text{ (no relay)}; \quad \times 2 \text{ relayed} \approx 22\% \text{ vs the } 25\% \text{ throttle}$$

The second figure in the equation records a subtlety of running a mesh protocol with exactly two nodes. Meshtastic's default role re-broadcasts every overheard packet to extend multi-hop meshes, but with two nodes every relay is pure waste that doubles the spend to roughly 22 percent, brushing the throttle precisely when both vehicles move. Both radios therefore run the CLIENT_MUTE role, which transmits and receives normally but never relays; the change was measured to halve idle utilization from the 9 to 13 percent range to 5 to 6 percent. The full cadence contract is: positions every 10 seconds and 20 meters while moving, every 60 seconds while parked, device telemetry every 300 seconds, and node information on boot. Each periodic packet exists to prevent a named failure, and the telemetry heartbeat earns the next section.

6.4 Liveness is arithmetic

No wire connects the vehicles, so nothing can prove a silent partner is alive. Every liveness indicator in the system is a comparison between the current time and a stored last-heard timestamp. The stock firmware's definition of an online node is one inequality, with the constant defined in its node database source (Meshtastic Project, *Firmware*):

$$\text{online}(n) \;\Leftrightarrow\; t_{now} - t_{last\ heard}(n) < 7200 \text{ s}$$

The two-hour window is sized for an ecosystem of solar nodes that legitimately sleep, and it must exceed the longest honest silence or healthy nodes flap offline; its cost is that a dead radio can look alive for two hours. Because this system's cadence contract guarantees a packet at least every 300 seconds, the display layers far stricter thresholds on the same timestamps: a link-fresh tag within 90 seconds, a visibly degraded position estimate past 30 seconds, and a stale-data banner past 120 seconds. Several of the semester's most confusing symptoms, including online counters claiming a partner was last heard 72 days ago while text messages flowed normally, were ultimately stale stored timestamps being re-broadcast by a node without a GNSS fix, and every one of them yielded to the same analysis: the counter is not a connection, it is arithmetic on a timestamp, so find the timestamp that is lying.

7. The Display Node: Waveshare ESP32-S3-RLCD-4.2

7.1 Hardware

The display node is a Waveshare ESP32-S3-RLCD-4.2 driver board: the same ESP32-S3 microcontroller class as the radio, driving a 4.2-inch, 400 by 300 pixel reflective LCD (Waveshare). A reflective panel modulates ambient light rather than emitting its own, which makes it brightest exactly where a dashboard instrument needs it, in direct sunlight, while drawing milliwatts and producing no glare at night. The board

exposes the single button and USB-C port the firmware uses, and both units are enclosed in PETG cases printed from a community design, chosen over PLA because dashboard temperatures exceed PLA's glass transition.

7.2 What made it possible: the Client API over BLE

The decision that made a custom display feasible alongside unmodified radio firmware is that Meshtastic publishes its phone-application interface as a documented Bluetooth Low Energy service, the Client API (Meshtastic Project, "Client API"). The radio exposes one GATT service with three characteristics: a ToRadio characteristic the client writes commands and outbound packets into, a FromRadio characteristic the client reads repeatedly until empty, and a FromNum characteristic that notifies when new data is queued. Every payload is a protocol buffer, a compact binary serialization with published schemas. The display firmware therefore impersonates the phone app: it connects as a BLE central using the NimBLE-Arduino stack (h2zero), writes a configuration request, drains the radio's node database, and then drains every subsequent notification, decoding each FromRadio message with nanopb, a protocol-buffer decoder designed for microcontrollers (Aimonen). The radio cannot tell the difference between this team's 400-line protocol layer and the official application, which is precisely why no modification of the radio firmware, and none of the risk that would carry, was ever required.

The decoded stream feeds a deliberately simple architecture: a Bluetooth task writes and a rendering task reads a single mutex-protected state structure containing both vehicles' fixes, names, battery levels, and message history. The one-writer, one-reader discipline eliminates data races by construction, and because the shared structure is pure data with no hardware dependencies, the navigation mathematics compiles and runs on a desktop computer, where twelve unit tests exercise it against known coordinates.

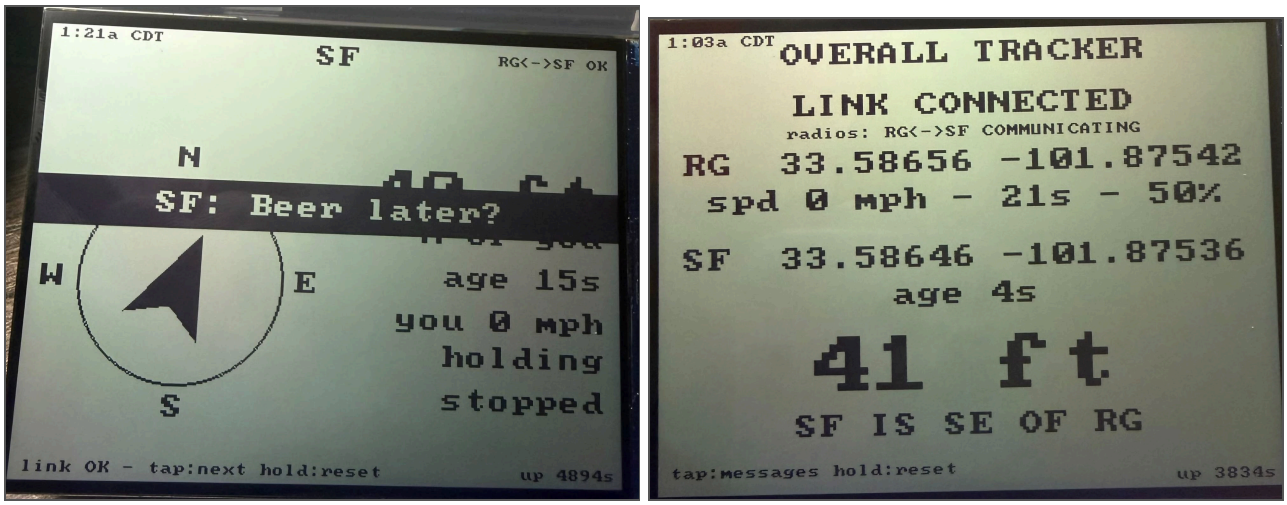


Figure 4. The display in operation: the compass page with an arriving message (left) and the status page listing both vehicles, their data ages, batteries, and separation (right).

8. Navigation Mathematics

Everything the driver sees is computed on the display from two stored fixes. Latitude φ and longitude λ are in radians, ψ is the vehicle's own course over ground, and R is the Earth's mean radius. The distance between the vehicles is the great-circle distance given by the haversine formula, whose intermediate quantity a measures the separation on the unit sphere (Sinnott):

$$a = \sin^2\frac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2\frac{\Delta\lambda}{2}, \qquad d = 2R \arcsin\sqrt{a}, \qquad R = 6371 \text{ km}$$

During bench verification the two radios, sitting on one desk, reported fixes that this formula placed 12.5 meters apart, which is not an error in the formula but an honest picture of consumer GNSS scatter between two independent receivers; the figure tightened outdoors. The direction to the partner is the initial great-circle bearing, and subtracting the driver's own course and wrapping to plus or minus 180 degrees gives the angle the on-screen arrow makes with the vehicle's nose:

$$\theta = \operatorname{atan2}(\sin\Delta\lambda \cos\varphi_2,\ \cos\varphi_1 \sin\varphi_2 - \sin\varphi_1 \cos\varphi_2 \cos\Delta\lambda), \qquad \alpha_{arrow} = \operatorname{wrap}_{\pm 180}(\theta - \psi_{course})$$

Radio updates arrive at most every ten seconds, but the screen renders twice per second. Between packets each fix is projected forward along its last course and speed, with the cosine of latitude correcting for the convergence of meridians, and with projection capped at 120 seconds:

$$\varphi(t) = \varphi_0 + \frac{v\,\Delta t \cos\psi}{R}, \qquad \lambda(t) = \lambda_0 + \frac{v\,\Delta t \sin\psi}{R\cos\varphi}, \qquad \Delta t \le 120 \text{ s}$$

The projection is cosmetic, and the design is careful to say so: the displayed data age is never reset by projection, and past 30 seconds the arrow is drawn hollow, so motion stays smooth while the symbology tells the truth about freshness. Finally, the display reports whether the gap is closing or opening through an exponential moving average of the distance derivative, sampled each 500-millisecond render tick; the 0.9 to 0.1 weighting settles in about five seconds, enough smoothing that GNSS jitter does not flap the label:

$$r_k = 0.9\, r_{k-1} + 0.1 \cdot \frac{d_{k-1} - d_k}{\Delta t}, \qquad \Delta t = 0.5 \text{ s}; \quad r > 0: \text{ closing}, \quad r < 0: \text{ apart}$$

9. Experimental Verification

Every major claim in this report has been exercised on the real hardware, and several of the system's rules exist only because an experiment failed first.

Claim	Evidence
Two-way link on slot 43	Round-trip traceroutes across multiple sessions, 5.75 to 7.25 dB SNR per direction at +30 dBm
Frequency-slot derivation	Both OLEDs display 912.875 MHz on channel Group9; the 2026-09-25 outage reproduced the mismatch case
Live tracking at vehicle cadence	A display tracked a real drive on first boot; the 10-second moving cadence held through the Week 4 drive test
On-demand position request	Both the command-line request and the display's button returned the partner's live fix within seconds
Messaging path	Preset texts delivered in both directions; the popup confirmed receiver-side only
Airtime management	Switching both nodes to CLIENT_MUTE halved measured idle utilization, from roughly 9 to 13 percent down to 5 to 6 percent
Navigation mathematics	Twelve host-side unit tests pass; bench distances match hand-computed haversine values

The remaining planned verification is a two-vehicle drive test with acceptance metrics declared in advance: a 95th-percentile partner-position age no greater than 15 seconds while moving, reacquisition within 60 seconds after a separation beyond range, and no silent data loss in convoy, followed by a range characterization plotting measured signal-to-noise ratio against distance for comparison with the Section 5 prediction.

10. Conclusion

The system works because each layer keeps one precise promise to the layer above it. Chirp spread spectrum promises that 11-bit symbols survive 17.5 dB below the noise floor, and charges 8.19 milliseconds per symbol for it. The link budget promises 161 dB of absorbable loss, and the design spends 20 dB of it on the real world to quote an honest 3 to 5 km. Meshtastic promises that a channel name, hashed, is a frequency, and that a key shared over a cable is privacy on public airspace. The Client API promises that anything that speaks its protocol buffers over three Bluetooth characteristics is a legitimate client, which is what admitted a custom sunlight-readable display without a line of radio firmware changing. And the display, holding the top of the stack, promises the driver honesty: every number on the glass is either a measurement with its age attached or an estimate drawn hollow. The result is two ordinary vehicles that know where each other are, anywhere on Earth with a view of the sky, with nothing between them but physics and agreement.

Works Cited

Aimonen, Petteri. *Nanopb: Protocol Buffers with Small Code Size*. GitHub, github.com/nanopb/nanopb. Accessed 4 Oct. 2026.

Espressif Systems. *ESP32-S3 Series Datasheet*. Version 2.0, Espressif Systems, 2024, www.espressif.com/en/products/socs/esp32-s3.

h2zero. *NimBLE-Arduino: A Fork of the NimBLE Stack for Arduino*. GitHub, github.com/h2zero/NimBLE-Arduino. Accessed 4 Oct. 2026.

Heltec Automation. *WiFi LoRa 32 (V4) Product Documentation*. Heltec Automation, heltec.org/project/wifi-lora-32-v4. Accessed 4 Oct. 2026.

Meshtastic Project. "Client API." *Meshtastic Documentation*, meshtastic.org/docs/development/device/client-api. Accessed 4 Oct. 2026.

Meshtastic Project. "Encryption." *Meshtastic Documentation*, meshtastic.org/docs/overview/encryption. Accessed 4 Oct. 2026.

Meshtastic Project. "Mesh Broadcast Algorithm." *Meshtastic Documentation*, meshtastic.org/docs/overview/mesh-algo. Accessed 4 Oct. 2026.

Meshtastic Project. "Radio Settings." *Meshtastic Documentation*, meshtastic.org/docs/configuration/radio. Accessed 4 Oct. 2026.

Meshtastic Project. *Meshtastic Device Firmware*. Version 2.7.26, GitHub, github.com/meshtastic/firmware. Accessed 4 Oct. 2026.

Semtech Corporation. *AN1200.13: LoRa Modem Designer's Guide*. Semtech Corporation, 2013, www.semtech.com.

Semtech Corporation. *SX1261/2 Long Range, Low Power, Sub-GHz RF Transceiver Datasheet*. Revision 2.2, Semtech Corporation, 2023, www.semtech.com/products/wireless-rf/lora-connect/sx1262.

Sinnott, Roger W. "Virtues of the Haversine." *Sky and Telescope*, vol. 68, no. 2, 1984, p. 159.

United States. *Code of Federal Regulations*, title 47, section 15.247. Electronic Code of Federal Regulations, www.ecfr.gov/current/title-47/section-15.247. Accessed 4 Oct. 2026.

Waveshare Electronics. *ESP32-S3-RLCD-4.2 Development Board Wiki*. Waveshare, www.waveshare.com/wiki/ESP32-S3-RLCD-4.2. Accessed 4 Oct. 2026.